NAME & REG NO

DHARM VIPULBHAI PATEL(24BCE2012)

OWAIS CHAUDHARY(24BCE0962)

MOHAMMAD AQUIB(24BCE0998)

Review 1

Hospital Appointment Booking System

Problem Definition, Requirement Analysis & Database Design

1. Problem Statement

In a hospital, managing patient appointments manually or through conventional systems can lead to problems such as appointment clashes, difficulty in maintaining patient and doctor records, inefficient scheduling, and delays in accessing appointment information.

The proposed system, Hospital Appointment Booking System, is designed to provide a centralized database-based solution for managing patients, doctors, departments, schedules, and appointments. The system allows patients to book available appointments with doctors, while hospital staff and doctors can manage appointment-related information efficiently.

The system will use MongoDB as the NoSQL database because the application involves document-oriented data, frequently changing information, appointment records, and queries that can benefit from aggregation and indexing.

2. Objectives

The main objectives of the Hospital Appointment Booking System are:

1. To maintain patient information in a structured and easily accessible manner.
2. To maintain doctor and department information.
3. To manage doctor availability and appointment schedules.
4. To allow patients to book, view, update, and cancel appointments.
5. To prevent duplicate or conflicting appointment bookings.
6. To provide efficient searching and retrieval of appointment information.

7. To support hospital staff in managing doctors, patients, and appointments.
8. To use MongoDB features such as documents, CRUD operations, aggregation, and indexing.
9. To provide a simple interface/dashboard for interacting with the system.
10. To provide basic authentication for different users.

3. Scope

The scope of the system includes:

- Patient registration and profile management.
- Doctor registration and profile management.
- Department management.
- Doctor availability and time-slot management.
- Appointment booking, modification, viewing, and cancellation.
- Searching appointments by patient, doctor, department, and date.
- Basic user authentication and role management.
- Dashboard for viewing appointment-related information.
- MongoDB-based storage and retrieval of application data.
- Use of aggregation and indexes for efficient queries.

The project focuses on appointment management and related hospital information. It does not cover complete hospital operations such as laboratory management, billing, pharmacy inventory, or emergency-response management.

4. Requirement Specification

4.1 Functional Requirements

A. User Management

- Users should be able to register and log in.
- The system should support different roles such as patient, doctor, and administrator.
- User information should be stored securely.
- Administrators should be able to manage user records.

B. Patient Management

- Patients should be able to create and update their profiles.
- The system should store patient details such as name, age, gender, contact information, and medical information.
- Patients should be able to view their appointment history.

C. Doctor Management

- The system should store doctor details such as name, specialization, department, contact information, and availability.
- Administrators should be able to add, update, and remove doctor records.
- Doctors should be able to view their scheduled appointments.

D. Department Management

- The system should maintain hospital departments.
- Doctors should be associated with departments.
- Patients should be able to search doctors based on department or specialization.

E. Appointment Management

- Patients should be able to book an available appointment.
- Patients should be able to view their upcoming and previous appointments.
- Patients should be able to cancel or reschedule an appointment.
- Doctors should be able to view their appointments.
- Administrators should be able to monitor and manage appointments.
- The system should prevent conflicting bookings.

F. Schedule Management

- Doctor availability should be stored in the database.
- Available time slots should be displayed to patients.
- Booked time slots should not be available for another appointment.

G. Dashboard

The system should provide a simple dashboard showing relevant information such as:

- Total patients

- Total doctors
- Available appointments
- Upcoming appointments
- Appointment status

H. Database Operations

The system should support:

- Create operations
- Read operations
- Update operations
- Delete operations
- Aggregation queries
- Referencing queries
- Indexed queries

4.2 Non-Functional Requirements

- The system should provide efficient data retrieval.
- The database should be scalable for increasing numbers of patients and appointments.
- The system should provide basic authentication.
- Frequently searched fields should be indexed.
- The interface should be simple and user-friendly.
- Data should be stored in a flexible document-oriented structure.
- The system should maintain data consistency for appointment booking.

5. Selected NoSQL Database: MongoDB

MongoDB is selected as the NoSQL database for the Hospital Appointment Booking System.

MongoDB is a document-oriented NoSQL database in which data is stored as JSON-like documents inside collections. This model is suitable for hospital appointment data because patient, doctor, appointment, and schedule information can be represented as flexible documents.

The provided project requirements specifically identify MongoDB as a suitable NoSQL option and associate it with hospital management using concepts such as documents, aggregation, and indexing.

6. Why MongoDB is Suitable

6.1 Document-Oriented Data Model

Hospital-related information contains multiple attributes that can naturally be represented as documents.

For example, a patient can be represented as a document containing:

- Patient ID
- Name
- Age
- Gender
- Contact information
- Address
- Medical information

This makes MongoDB's document model suitable for the system.

6.2 Flexible Schema

Hospital information can change over time. For example, additional patient or doctor information may be introduced later.

MongoDB provides a flexible schema, allowing documents to contain different fields when required without forcing the entire database structure to be redesigned.

6.3 Aggregation

The system requires appointment-related analysis such as:

- Number of appointments per doctor
- Number of appointments per department
- Daily appointment count
- Completed versus cancelled appointments
- Number of appointments for a particular patient

MongoDB's aggregation framework is suitable for performing these operations.

6.4 Indexing

The system will frequently search information using fields such as:

- patient_id
- doctor_id
- department_id
- appointment_date
- appointment_status

Indexes can be created on these fields to improve query performance.

6.5 CRUD Operations

MongoDB supports all required CRUD operations:

- Create — adding patients, doctors, and appointments
- Read — viewing patient and appointment information
- Update — changing profiles or appointment details
- Delete — removing or cancelling records

6.6 Scalability

The number of patients and appointments can increase significantly in a hospital environment. MongoDB is suitable for handling large volumes of document-oriented data and can scale as the application grows.

7. Justification Compared with Other NoSQL Models

MongoDB is preferred over Cassandra, Neo4j, and Redis for this project because the primary requirement is management of structured but flexible hospital documents and appointment records.

Neo4j would be more appropriate when the main focus is complex relationship traversal, Cassandra when handling large-scale time-series or highly distributed data, and Redis when fast in-memory key-value operations are the primary requirement.

For this particular project, MongoDB provides the required combination of:

- Document storage
- CRUD operations
- Aggregation
- Indexing
- Flexible schema
- Referencing

- Scalability

Therefore, MongoDB is an appropriate NoSQL database for the Hospital Appointment Booking System.

8. Data Model Design

The Hospital Appointment Booking System will use six MongoDB collections:

1. users
2. patients
3. doctors
4. departments
5. appointments
6. schedules

This satisfies the project requirement of having 4–6 collections.

9. Collection 1: Users

The users collection stores authentication and role-related information.

Schema

```
users
-------------------------
_id
name
email
password
role
phone
created_at
```

Role Values

```
patient
doctor
admin
```

Purpose

This collection is used for authentication and authorization of different users in the system.

10. Collection 2: Patients

The patients collection stores patient-specific information.

Schema

```
patients
------------------------
_id
user_id
patient_name
age
gender
phone
address
blood_group
created_at
```

Reference

```
user_id → users._id
```

A patient is associated with a user account through `user_id`.

11. Collection 3: Doctors

The doctors collection stores information about doctors.

Schema

```
doctors
------------------------
_id
user_id
doctor_name
specialization
department_id
phone
experience
consultation_fee
```

References

```
user_id → users._id
department_id → departments._id
```

Each doctor belongs to a particular department.

12. Collection 4: Departments

The departments collection stores hospital department information.

Schema

```
departments
-------------------------
_id
department_name
description
location
```

Example Departments

```
Cardiology
Neurology
Orthopedics
Dermatology
General Medicine
```

A department can have multiple doctors.

13. Collection 5: Appointments

The appointments collection is the central collection of the system.

Schema

```
appointments
-------------------------
_id
patient_id
doctor_id
department_id
schedule_id
appointment_date
appointment_time
reason
status
created_at
```

References

```
patient_id → patients._id
doctor_id → doctors._id
department_id → departments._id
schedule_id → schedules._id
```

Status Values

```
Booked
Completed
Cancelled
Rescheduled
```

This collection connects patients, doctors, departments, and available schedules.

14. Collection 6: Schedules

The schedules collection stores doctor availability.

Schema

```
schedules
-------------------------
_id
doctor_id
date
start_time
end_time
slot_status
```

Reference

```
doctor_id → doctors._id
```

Slot Status

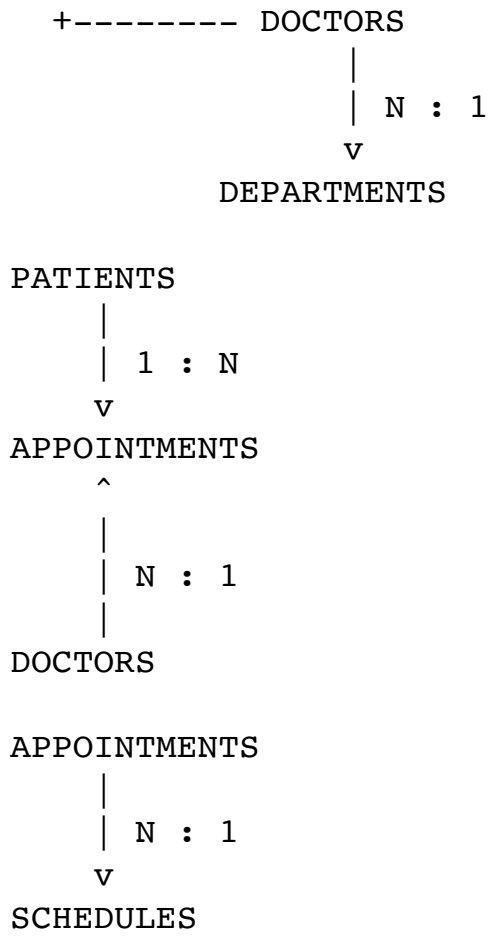
```
Available
Booked
Unavailable
```

This collection allows the system to identify available appointment slots.

15. Overall Relationship / ER-to-NoSQL Mapping

The logical relationships can be represented as:

```
USERS
  |
  | 1 : 1
  |
  +-------- PATIENTS
  |
```



Relationship Explanation

- One user account can correspond to one patient or doctor profile.
- One department can contain multiple doctors.
- One patient can have multiple appointments.
- One doctor can have multiple appointments.
- One doctor can have multiple schedules.
- Each appointment is associated with one patient, one doctor, one department, and one schedule.

The relational entities are mapped into separate MongoDB collections, while relationships are maintained using document references such as `patient_id`, `doctor_id`, and `department_id`.

16. Sample Dataset Design

users

```
{
```

```
  "_id": "U001",
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "password": "hashed_password",
  "role": "patient",
  "phone": "9876543210",
  "created_at": "2026-08-20"
}
{
  "_id": "U002",
  "name": "Dr. Ananya Mehta",
  "email": "ananya@example.com",
  "password": "hashed_password",
  "role": "doctor",
  "phone": "9876543211",
  "created_at": "2026-08-20"
}
```


patients


```
{
  "_id": "P001",
  "user_id": "U001",
  "patient_name": "Rahul Sharma",
  "age": 22,
  "gender": "Male",
  "phone": "9876543210",
  "address": "Delhi",
  "blood_group": "B+",
  "created_at": "2026-08-20"
}
```


departments


```
{
  "_id": "D001",
  "department_name": "Cardiology",
  "description": "Diagnosis and treatment of heart-related
conditions",
  "location": "Block A, First Floor"
}
```


doctors


```
{
  "_id": "DOC001",
  "user_id": "U002",
```

```
  "doctor_name": "Dr. Ananya Mehta",
  "specialization": "Cardiologist",
  "department_id": "D001",
  "phone": "9876543211",
  "experience": 8,
  "consultation_fee": 800
}
```

schedules

```
{
  "_id": "S001",
  "doctor_id": "DOC001",
  "date": "2026-08-25",
  "start_time": "10:00",
  "end_time": "10:30",
  "slot_status": "Available"
}
```

appointments

```
{
  "_id": "A001",
  "patient_id": "P001",
  "doctor_id": "DOC001",
  "department_id": "D001",
  "schedule_id": "S001",
  "appointment_date": "2026-08-25",
  "appointment_time": "10:00",
  "reason": "Regular heart check-up",
  "status": "Booked",
  "created_at": "2026-08-22"
}
```

17. Summary of Review 1 Deliverables

Review 1 Requirement	Included
Problem statement	Yes
Objectives	Yes
Scope	Yes
Requirement specification	Yes
NoSQL database selection	MongoDB

Justification of database	Yes
Data model design	Yes
Collections	6
ER-to-NoSQL mapping	Yes
Node/relationship design	Not applicable as primary model; references used
Sample dataset design	Yes

This design aligns with the project requirements given in the provided DBMS project document, including the use of multiple collections, CRUD-oriented application structure, aggregation, indexing, authentication, a simple interface/dashboard, and documentation of the schema and NoSQL model choice.